Ad Soyad: _____	**Tarih:** _____
Bölüm 9 – Ünite 3 – *std::mutex ve Senkronizasyon – Çalışma Kağıdı*	

Süre

Bu çalışma kağıdını tamamlamak için tahmini süre: **45–50 dakika**.

Bölüm 1: Boşluk Doldurma

(5 dakika – 10 puan)

Aşağıdaki cümlelerdeki boşlukları uygun kavram ile doldurun.

1. Birden fazla izleğin paylaşılan veriye eş zamanlı erişip en az birinin yazma yapması durumuna _____ denir ve bu durum _____ davranışa yol açar.
2. Paylaşılan veriye güvenli erişim sağlamak için yalnızca bir izleğin girebildiği kod bölgesine _____ adı verilir.
3. `std::lock_guard` nesnesi oluşturulduğunda mutex'i _____ eder, kapsam sonunda yıkıcısı mutex'i otomatik olarak _____ eder.
4. İki izleğin birbirinin tuttuğu kilidi beklemesi sonucu her ikisinin de sonsuza dek bloklanmasına _____ denir.
5. C++17 ile gelen `std::scoped_lock` birden fazla mutex'i _____ şekilde kilitleyerek deadlock'u önler.

Bölüm 2: Eşleştirme

(5 dakika – 10 puan)

Sol sütundaki kilit yöneticisini sağ sütundaki özelliğiyle eşleştirin.

#	Kilit Yöneticisi	#	Özellik
1	`std::lock_guard`	a	Birden fazla mutex'i atomik olarak kilitler (C++17)
2	`std::unique_lock`	b	Yalnızca RAII kilitleme/açma; taşınamaz, esnetilemez
3	`std::scoped_lock`	c	Manuel `lock()`/`unlock()` ve `condition_variable` ile kullanım
4	`std::mutex::lock()`	d	RAII sarmalayıcısı olmadan doğrudan kilitleme; `unlock()` unutulma riski
5	`std::defer_lock`	e	`unique_lock` oluşturulur ama mutex henüz kilitlenmez

Cevaplar: 1→_____ 2→_____ 3→_____ 4→_____ 5→_____

Bölüm 3: Kod Analizi – Data Race ve Mutex

(10 dakika – 20 puan)

Aşağıdaki kodda bir data race sorunu vardır. Kodu inceleyin ve soruları cevaplayın.

```cpp
#include <iostream>
#include <thread>
#include <vector>

int sayac = 0;

void arttir(int n) {
    for (int i = 0; i < n; ++i)
        ++sayac;  // data race!
}

int main() {
    std::thread t1(arttir, 100000);
    std::thread t2(arttir, 100000);

    t1.join();
    t2.join();

    std::cout << "Sayac: " << sayac << std::endl;
    return 0;
}
```

1. Program her çalıştırıldığında `sayac` değeri 200000 olur mu? Nedenini açıklayın. (5 puan)

2. Bu sorunu `std::mutex` ve `std::lock_guard` kullanarak düzeltin. Eklemeniz/değiştirmeniz gereken satırları yazın. (5 puan)

3. `lock_guard` yerine mutex'i doğrudan `lock()`/`unlock()` ile yönetmenin sakıncası nedir? (5 puan)

4. Bu sorunu mutex yerine `std::atomic<int>` ile çözmek mümkün mü? Kısaca açıklayın. (5 puan)

Bölüm 4: Deadlock Senaryosu Analizi

(10 dakika – 20 puan)

Aşağıdaki kodda potansiyel bir deadlock bulunmaktadır. Kodu inceleyin.

```
#include <iostream>
#include <thread>
#include <mutex>

std::mutex m1, m2;

void islemA() {
    std::lock_guard<std::mutex> kilit1(m1);
    std::this_thread::sleep_for(
        std::chrono::milliseconds(10));
    std::lock_guard<std::mutex> kilit2(m2);
    std::cout << "A tamamlandi\n";
}

void islemB() {
    std::lock_guard<std::mutex> kilit1(m2);
    std::this_thread::sleep_for(
        std::chrono::milliseconds(10));
    std::lock_guard<std::mutex> kilit2(m1);
    std::cout << "B tamamlandi\n";
}

int main() {
    std::thread tA(islemA);
    std::thread tB(islemB);
    tA.join();
    tB.join();
    return 0;
}
```

1. Deadlock neden oluşuyor? Adım adım açıklayın. (7 puan)

2. Bu deadlock'u `std::scoped_lock` kullanarak nasıl çözersiniz? `islemA` ve `islemB` fonksiyonlarını yeniden yazın. (7 puan)

3. `std::scoped_lock` yerine kilitleme sırasını (lock ordering) düzelterek de çözülebilir mi? Nasıl? (6 puan)

Bölüm 5: Açık Uçlu Sorular

(10 dakika – 20 puan)

1. OBYS projesinde birden fazla izlek aynı öğrenci listesine ekleme yapmak istediğinde hangi senkronizasyon stratejisini kullanırsınız? `lock_guard` mı, `unique_lock` mu? Gerekçenizi yazın. (10 puan)

2. "Mümkün olan en kısa süre boyunca kilidi tut" ilkesini (fine-grained locking) bir OBYS senaryosuyla açıklayın. Kaba taneli (coarse-grained) kilitleme ile karşılaştırın. (10 puan)